

A Practical Course on Mathematical Modelling

Lecture 4 - Modelling Tricks with Integer Variables

Prof. Viet Hung Nguyen

Clermont Auvergne University - Clermont-Ferrand - France

LOGIC, ACTIVATION, DISJUNCTIONS AND LINEARIZATION

Part I — Binary decisions and activation

Yes–no decisions → fixed charges → semicontinuous variables → conditional constraints → indicators.

Part II — Logic, counting and disjunctions

AND / OR / XOR → at most / at least / exactly k → either–or constraints.

Part III — Linearization tricks

Products → absolute values → maxima → piecewise-linear functions → good practice.

Represent a Yes–No Decision

PART I · THE BINARY VARIABLE

Let $y \in \{0, 1\}$ represent a statement or a discrete decision:

$y = 1 \iff$ the decision is selected or the statement is true.

Typical meanings

- open a facility;
- use a machine;
- select a project;
- satisfy a condition;
- choose one branch of a disjunction.

The modelling principle

Translate the truth table or the intended operating modes into linear inequalities.

Bounds on continuous variables are often essential.

Example: Warehouse-Opening Decisions

PART I · FROM REQUIREMENTS TO INEQUALITIES

A company may open three warehouses. Define

$$y_j = \begin{cases} 1, & \text{if warehouse } j \text{ is opened,} \\ 0, & \text{otherwise,} \end{cases} \quad j = 1, 2, 3.$$

Then common requirements become linear constraints:

$$\begin{aligned} y_1 + y_2 + y_3 &\leq 2 && \text{open at most two warehouses,} \\ y_1 &\leq y_2 && \text{opening 1 requires opening 2,} \\ y_2 + y_3 &\geq 1 && \text{open warehouse 2 or 3.} \end{aligned}$$

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, y[1:3], Bin)
@constraint(model, sum(y) <= 2)      # at most two
@constraint(model, y[1] <= y[2])    # 1 requires 2
@constraint(model, y[2] + y[3] >= 1) # 2 or 3
@objective(model, Max, 8y[1] + 7y[2] + 5y[3])
optimize!(model)
value.(y)
```

Pay a Cost Only When an Activity Starts

PART I · FIXED-CHARGE LINKING

Producing $x \geq 0$ costs cx plus a fixed charge F whenever $x > 0$.

$$\min \quad cx + Fy \quad 0 \leq x \leq Uy, \quad y \in \{0, 1\}.$$

If $y = 0$, then $x = 0$: production is closed.

If $y = 1$, then $0 \leq x \leq U$: production is allowed.

Valid upper bound

U must be a valid upper bound on x . A tight U gives a stronger LP relaxation.

Example: Machine Setup Cost

PART I · A FIXED CHARGE IN EUROS

A machine produces x units. Starting it costs 100 euros and each unit costs 3 euros. Its capacity is 40 units.

$$\min 3x + 100y, \quad 0 \leq x \leq 40y, \quad y \in \{0, 1\}.$$

- If $y = 0$, then $x = 0$ and no setup cost is paid.
- If $x > 0$, then $y = 1$ and the 100-euro charge is paid.

Minimum lot size. If at least 10 units must be produced after setup, replace the linking constraint by

$$10y \leq x \leq 40y.$$

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, x >= 0)
@variable(model, y, Bin)
@constraint(model, x <= 40y)
@constraint(model, x >= 25) # required demand
@objective(model, Min, 3x + 100y)
optimize!(model)
(value(x), value(y), objective_value(model))
```

The demand constraint is added to avoid the trivial zero-production solution.

An Activity That Is Either Off or Bounded

PART I · SEMICONTINUOUS VARIABLES

A machine is either idle or operates between L and U units:

$$x \in \{0\} \cup [L, U].$$

x is called a **semicontinuous** variable. An exact MILP formulation is

$$Ly \leq x \leq Uy, \quad y \in \{0, 1\}.$$

- $y = 0$ forces $x = 0$.
- $y = 1$ forces $L \leq x \leq U$.

Example: Operate a Bounded Generator

PART I · STOPPED OR BETWEEN 20 AND 100 MW

A generator is either stopped or produces between 20 and 100 MW:

$$x \in \{0\} \cup [20, 100].$$

Introduce $y \in \{0, 1\}$ and impose

$$20y \leq x \leq 100y.$$

y	Resulting domain of x	Operating state
0	$x = 0$	stopped
1	$20 \leq x \leq 100$	operating

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, x >= 0)
@variable(model, y, Bin)
@constraint(model, 20y <= x)
@constraint(model, x <= 100y)
@objective(model, Max, x)
optimize!(model)
(value(x), value(y))
```

Enforce a Constraint Only If a Decision Is Selected

PART I · THE BIG- M INEQUALITY

To model

$$y = 1 \implies a^\top x \leq b,$$

use the big- M inequality

$$a^\top x \leq b + M(1 - y).$$

- If $y = 1$, the original constraint is enforced.
- If $y = 0$, it is relaxed by M .

Valid value of M

$M \geq \max\{a^\top x - b : x \in X\}$, where X contains the other known bounds and constraints.

Example: Contract Capacity Limit

PART I · CHOOSING M FROM THE BOUNDS

Let $0 \leq x \leq 10$. If a particular contract is selected ($y = 1$), production must satisfy

$$x \leq 4.$$

Because the largest possible violation is $10 - 4 = 6$, choose $M = 6$:

$$x \leq 4 + 6(1 - y).$$

- If $y = 1$: $x \leq 4$.
- If $y = 0$: $x \leq 10$, so the additional limit disappears.

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, 0 <= x <= 10)
@variable(model, y, Bin)
@constraint(model, x <= 4 + 6 * (1 - y))
@constraint(model, y == 1) # selected-contract case
@objective(model, Max, x)
optimize!(model)
(value(x), value(y)) # x = 4, y = 1
```

Identify Whether a Condition Holds

PART I · ENCODING AN EQUIVALENCE, BOTH DIRECTIONS

To model $y = 1 \iff a^\top x \leq b$ one must encode both directions. With a tolerance or discrete gap $\varepsilon > 0$:

$$\begin{aligned}a^\top x &\leq b + M^+(1 - y), \\a^\top x &\geq b + \varepsilon - M^-y.\end{aligned}$$

With $U_a = \max_{x \in X} a^\top x$ and $L_a = \min_{x \in X} a^\top x$, the tight deactivation constants are

$$\begin{aligned}M^+ &= \max\{0, U_a - b\}, \\M^- &= \max\{0, b + \varepsilon - L_a\}.\end{aligned}$$

M^+ relaxes the upper bound when $y = 0$; M^- relaxes the lower bound when $y = 1$.

The gap ε must be meaningful for the application.

Example: Is a Threshold Respected?

PART I · AN INTEGER VARIABLE AND $\varepsilon = 1$

Suppose x is integer, $0 \leq x \leq 10$, and

$$y = 1 \iff x \leq 4.$$

Because x is integer, take $\varepsilon = 1$. Then

$$L_a = 0, \quad U_a = 10, \quad M^+ = 10 - 4 = 6, \quad M^- = 4 + 1 - 0 = 5.$$

The formulation is

$$x \leq 4 + 6(1 - y), \quad x \geq 5 - 5y.$$

If $y = 1$, then $x \leq 4$; if $y = 0$, then $x \geq 5$.

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, 0 <= x <= 10, Int)
@variable(model, y, Bin)
@constraint(model, x <= 4 + 6 * (1 - y))
@constraint(model, x >= 5 - 5y)
@objective(model, Max, x)
optimize!(model)
(value(x), value(y)) # y = 1 iff x <= 4
```

Identify When All Conditions Are Satisfied

PART II · LOGICAL AND

Let $z = 1$ if and only if $x_1 = x_2 = 1$:

$$z = x_1 \wedge x_2.$$

The linear formulation is

$$z \leq x_1, \quad z \leq x_2, \quad z \geq x_1 + x_2 - 1, \quad z \in \{0, 1\}.$$

For n binary variables

$$z \leq x_i \quad (i = 1, \dots, n), \quad z \geq \sum_{i=1}^n x_i - (n - 1).$$

Example: A Bonus for Selecting Both Projects

PART II · AND IN THE OBJECTIVE

Projects 1 and 2 generate an additional profit of 20 only when both are selected. Let $x_1, x_2 \in \{0, 1\}$ and introduce $z = x_1 \wedge x_2$:

$$z \leq x_1, \quad z \leq x_2, \quad z \geq x_1 + x_2 - 1.$$

Add $20z$ to the objective. The possible values are

(x_1, x_2)	(0, 0)	(0, 1)	(1, 0)	(1, 1)
z	0	0	0	1

Thus the bonus is obtained only in the final case.

JuMP: bonus for selecting both projects

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, x[1:2], Bin)
@variable(model, z, Bin)
@constraint(model, z <= x[1])
@constraint(model, z <= x[2])
@constraint(model, z >= x[1] + x[2] - 1)
@objective(model, Max, 12x[1] + 12x[2] + 20z)
optimize!(model)
(value.(x), value(z))
```

Identify When at Least One Condition Is Satisfied

PART II · LOGICAL OR

Let $z = 1$ if at least one of x_1, x_2 equals one:

$$z = x_1 \vee x_2.$$

The linear formulation is

$$z \geq x_1, \quad z \geq x_2, \quad z \leq x_1 + x_2, \quad z \in \{0, 1\}.$$

For n binary variables

$$z \geq x_i \quad (i = 1, \dots, n), \quad z \leq \sum_{i=1}^n x_i.$$

Example: Detect Available Backup Equipment

PART II · OR AS A RELIABILITY INDICATOR

Let x_1 and x_2 indicate whether two backup generators are installed. Define $z = 1$ when at least one generator is available:

$$z \geq x_1, \quad z \geq x_2, \quad z \leq x_1 + x_2.$$

(x_1, x_2)	(0, 0)	(0, 1)	(1, 0)	(1, 1)
z	0	1	1	1

A reliability requirement such as “backup must be available” is then simply $z = 1$.

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, x[1:2], Bin)
@variable(model, z, Bin)
@constraint(model, z >= x[1])
@constraint(model, z >= x[2])
@constraint(model, z <= x[1] + x[2])
@constraint(model, z == 1)
@objective(model, Min, 2x[1] + 3x[2])
optimize!(model)
value.(x)
```

Opposite or Mutually Exclusive Decisions

PART II · NOT, IMPLIES, EQUIVALENT, XOR

For binary variables x, y, z :

Logical relation	Meaning	Linear formulation
$z = \neg x$	opposite values	$z = 1 - x$
$x \Rightarrow y$	precedence	$x \leq y$
$x \Leftrightarrow y$	same value	$x = y$
$z = x \oplus y$	exactly one true	$z = x + y - 2w$

XOR. $w = x \wedge y$ is binary and satisfies

$$w \leq x, \quad w \leq y, \quad w \geq x + y - 1.$$

Example: Choose Exactly One Transport Mode

PART II · XOR AND IMPLICATION COMBINED

Let $x_R = 1$ for road transport and $x_T = 1$ for rail transport. Exactly one mode must be selected:

$$x_R + x_T = 1.$$

This single equality is an XOR relation:

$$(x_R, x_T) \in \{(1, 0), (0, 1)\}.$$

If road transport also requires insurance, represented by y , add the implication

$$x_R \leq y.$$

If insurance is purchased exactly when road is chosen, write $x_R = y$.

JuMP: exactly one transport mode

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, road, Bin)
@variable(model, rail, Bin)
@variable(model, insured, Bin)
@constraint(model, road + rail == 1)
@constraint(model, road <= insured) # road implies insurance
@constraint(model, road == 1)      # inspect road mode
@objective(model, Min, insured)
optimize!(model)
(value(road), value(rail), value(insured))
```

Choose a Prescribed Number of Alternatives

PART II · CARDINALITY CONSTRAINTS

For binary variables x_i :

$$\text{at most } k : \sum_i x_i \leq k,$$

$$\text{at least } k : \sum_i x_i \geq k,$$

$$\text{exactly } k : \sum_i x_i = k.$$

These constraints are often combined with implications or incompatibilities.

Example: Form a Project Portfolio

PART II · CARDINALITY, INCOMPATIBILITY, CONDITIONAL COUNT

Five projects are available, with $x_i = 1$ if project i is selected.

Choose exactly three projects

$$\sum_{i=1}^5 x_i = 3.$$

Projects 1 and 2 are incompatible

$$x_1 + x_2 \leq 1.$$

If project 5 is selected, at least two of the other projects must be selected

$$\sum_{i=1}^4 x_i \geq 2x_5.$$

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, x[1:5], Bin)
@constraint(model, sum(x) == 3)
@constraint(model, x[1] + x[2] <= 1)
@constraint(model, sum(x[1:4]) >= 2x[5])
profit = [4, 3, 5, 2, 10]
@objective(model, Max, sum(profit[i] * x[i] for i in 1:5))
optimize!(model)
value.(x)
```

Require at Least One of Two Constraints to Hold

PART II · A FIRST DISJUNCTION

Suppose at least one of the following must hold:

$$a^{\top}x \leq b \quad \text{or} \quad c^{\top}x \leq d.$$

Introduce $y \in \{0, 1\}$ and write

$$\begin{aligned} a^{\top}x &\leq b + M_1y, \\ c^{\top}x &\leq d + M_2(1 - y). \end{aligned}$$

If $y = 0$, the first constraint is active; if $y = 1$, the second is active.

At least one branch

This enforces at least one branch. Both branches may hold simultaneously.

Example: Avoid a Forbidden Interval

PART II · EITHER $x \leq 2$ OR $x \geq 8$

Let $0 \leq x \leq 10$. We require

$$x \leq 2 \quad \text{or} \quad x \geq 8.$$

Introduce $y \in \{0, 1\}$:

$$x \leq 2 + 8y, \quad x \geq 8y.$$

- If $y = 0$, the first inequality gives $x \leq 2$.
- If $y = 1$, the second inequality gives $x \geq 8$.

Thus no feasible solution can satisfy $2 < x < 8$.

JuMP: avoid a forbidden interval

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, 0 <= x <= 10)
@variable(model, y, Bin)
@constraint(model, x <= 2 + 8y)
@constraint(model, x >= 8y)
@objective(model, Max, x)
optimize!(model)
(value(x), value(y)) # x = 10
```

Apply a Term Only When Two Decisions Are Selected

PART III · PRODUCT OF TWO BINARIES

To replace the nonlinear term

$$z = xy, \quad x, y \in \{0, 1\},$$

use

$$z \leq x, \quad z \leq y, \quad z \geq x + y - 1, \quad z \geq 0.$$

Here z may be declared continuous: the inequalities and binary values of x, y already force $z \in \{0, 1\}$.

Example: Pairwise Interaction Cost

PART III · LINEARIZING $30x_1x_2$

Opening facilities 1 and 2 simultaneously creates an additional coordination cost of 30:

$$30x_1x_2.$$

Introduce a continuous variable $z \geq 0$ and impose

$$z \leq x_1, \quad z \leq x_2, \quad z \geq x_1 + x_2 - 1.$$

Replace the nonlinear term by $30z$. Since $z = 1$ exactly when $x_1 = x_2 = 1$, the cost is charged in precisely that case.

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, x[1:2], Bin)
@variable(model, 0 <= z <= 1)
@constraint(model, z <= x[1])
@constraint(model, z <= x[2])
@constraint(model, z >= x[1] + x[2] - 1)
@objective(model, Max, 40x[1] + 40x[2] - 30z)
optimize!(model)
(value.(x), value(z))
```

Keep a Quantity Only When an Option Is Selected

PART III · BINARY $\times$ BOUNDED CONTINUOUS

Let $y \in \{0, 1\}$, $L \leq x \leq U$, and $z = yx$. The exact linearization is

$$\begin{aligned} z &\geq Ly, & z &\leq Uy, \\ z &\geq x - U(1 - y), & z &\leq x - L(1 - y). \end{aligned}$$

- If $y = 0$, the first pair forces $z = 0$.
- If $y = 1$, the second pair forces $z = x$.

Special case. If $0 \leq x \leq U$, this simplifies to

$$0 \leq z \leq Uy, \quad z \leq x, \quad z \geq x - U(1 - y).$$

Example: Sales Counted Only During a Promotion

PART III · $z = yx$ **WITH** $L = 0$, $U = 10$

Let $0 \leq x \leq 10$ be the number of units sold and $y = 1$ if a promotion is active. We need $z = yx$, the promoted sales. Because $L = 0$ and $U = 10$, impose

$$0 \leq z \leq 10y, \quad z \leq x, \quad z \geq x - 10(1 - y).$$

For $x = 7$:

$$y = 0 \Rightarrow z = 0, \quad y = 1 \Rightarrow z = 7.$$

The nonlinear product does not appear in the MILP.

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, 0 <= x <= 10)
@variable(model, y, Bin)
@variable(model, z >= 0)
@constraint(model, x == 7)
@constraint(model, z <= 10y)
@constraint(model, z <= x)
@constraint(model, z >= x - 10 * (1 - y))
@objective(model, Max, z)
optimize!(model)
(value(x), value(y), value(z))
```

Measure an Unsigned Deviation

PART III · LINEARIZING AN ABSOLUTE VALUE

To minimize or upper-bound $|x|$, introduce $t \geq 0$:

$$t \geq x, \quad t \geq -x.$$

Then minimizing t forces $t = |x|$.

Caveat

Without minimization or another upper pressure on t , these inequalities give only $t \geq |x|$, not equality. In this case, we will need extra binary variables.

Example: Minimize the Deviation from a Target

PART III · $|q - 100|$

Let q be production and let the target be 100 units. We want to minimize

$$|q - 100|.$$

Introduce $t \geq 0$ and write

$$t \geq q - 100, \quad t \geq 100 - q, \quad \min t.$$

For example:

$$q = 92 \Rightarrow t = 8, \quad q = 105 \Rightarrow t = 5$$

at an optimal solution for the chosen value of q .

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, 0 <= q <= 200)
@variable(model, t >= 0)
@constraint(model, q == 92) # use 105 for the second case
@constraint(model, t >= q - 100)
@constraint(model, t >= 100 - q)
@objective(model, Min, t)
optimize!(model)
(value(q), value(t)) # q = 92 gives t = 8
```

Represent the Largest or Smallest of Several Quantities

PART III · EPIGRAPH INEQUALITIES AND EXACT MAXIMUM

To model $z = \max\{x_1, \dots, x_n\}$ in a minimization context:

$$z \geq x_i \quad (i = 1, \dots, n).$$

Minimizing z gives equality automatically.

Equality independent of the objective. Introduce binary variables y_i representing the selection of x_i as the maximum:

$$z \geq x_i, \quad z \leq x_i + M_i(1 - y_i), \quad \sum_i y_i = 1.$$

Example: Minimize the Makespan

PART III · THE LAST MACHINE TO FINISH

Two machines finish their work at times C_1 and C_2 . The makespan is

$$C_{\max} = \max\{C_1, C_2\}.$$

Introduce z and solve

$$\min z \quad \text{s.t.} \quad z \geq C_1, \quad z \geq C_2.$$

If $C_1 = 7$ and $C_2 = 11$, the smallest feasible value is $z = 11$, exactly the completion time of the last machine.

JuMP: minimize the makespan

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
model = Model(HiGHS.Optimizer)
@variable(model, C1 == 7)
@variable(model, C2 == 11)
@variable(model, Cmax)
@constraint(model, Cmax >= C1)
@constraint(model, Cmax >= C2)
@objective(model, Min, Cmax)
optimize!(model)
value(Cmax) # 11
```

Cost or Response That Changes by Segments

PART III · PIECEWISE-LINEAR FUNCTIONS

For breakpoints $(\bar{x}_k, \bar{f}_k)$, a convex combination uses

$$x = \sum_k \lambda_k \bar{x}_k, \quad f = \sum_k \lambda_k \bar{f}_k, \quad \sum_k \lambda_k = 1, \quad \lambda_k \geq 0.$$

Additional adjacency restrictions are required for a general piecewise-linear graph:

- SOS2 constraints, if supported by the solver;
- binary segment-selection variables;
- incremental formulations.

Example: Shipping Quantity Discounts

PART III · THREE BREAKPOINTS AND AN SOS2 RESTRICTION

The shipping cost has breakpoints

$$(\bar{x}_k, \bar{f}_k) \in \{(0, 0), (10, 50), (20, 80)\}.$$

Use

$$x = 10\lambda_2 + 20\lambda_3, \quad f = 50\lambda_2 + 80\lambda_3, \quad \lambda_1 + \lambda_2 + \lambda_3 = 1.$$

An SOS2 restriction permits positive weights only on adjacent breakpoints. For $x = 15$, interpolation between $(10, 50)$ and $(20, 80)$ gives

$$\lambda_2 = \lambda_3 = 0.5, \quad f = 65.$$

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
xhat, fhat = [0.0, 10.0, 20.0], [0.0, 50.0, 80.0]
model = Model(HiGHS.Optimizer)
@variable(model, 0 <= lambda[1:3] <= 1)
@variable(model, x)
@variable(model, f)
@constraint(model, sum(lambda) == 1)
@constraint(model, x == sum(xhat[i]*lambda[i] for i in 1:3))
@constraint(model, f == sum(fhat[i]*lambda[i] for i in 1:3))
@constraint(model, lambda in SOS2())
@constraint(model, x == 15)
@objective(model, Min, f)
optimize!(model)
(value(f), value.(lambda)) # f = 65
```

JuMP reformulates SOS2 for HiGHS using a mixed-integer linear model.

Correct and Efficient Formulations

GOOD PRACTICE VERSUS COMMON MISTAKES

Good practice

- derive tight variable bounds;
- compute a separate M for each constraint;
- use indicator or perspective formulations when appropriate;
- exploit solver-supported SOS and semicontinuous variables.

Common mistakes

- choosing an arbitrary huge M ;
- encoding only one direction of an equivalence;
- forgetting the role of the objective;
- attempting to model a strict inequality;
- omitting valid bounds on continuous variables.

Example: A Correct Big- M Can Still Be Weak

TIGHT VERSUS LOOSE CONSTANTS

Consider

$$0 \leq x \leq 10, \quad y = 1 \Rightarrow x \leq 4.$$

The tight formulation uses $M = 6$:

$$x \leq 4 + 6(1 - y).$$

At the fractional LP point $y = 0.5$, this gives $x \leq 7$. Using $M = 10^6$ instead gives

$$x \leq 500004,$$

which adds no useful restriction beyond $x \leq 10$.

Both formulations are integer-valid, but the first provides a much stronger relaxation.

JuMP: tight versus loose M

JUMP AND HIGHS IMPLEMENTATION

```
using JuMP, HiGHS
for M in (6.0, 1.0e6)
    model = Model(HiGHS.Optimizer)
    @variable(model, 0 <= x <= 10)
    @variable(model, 0 <= y <= 1) # relaxed binary
    @constraint(model, y == 0.5)
    @constraint(model, x <= 4 + M * (1 - y))
    @objective(model, Max, x)
    optimize!(model)
    println("M = $M, x = ", value(x)) # 7, then 10
end
```

Choose a Construction for Each Modelling Need

PUTTING IT TOGETHER

Modelling need	Basic construction
Pay a fixed charge	$0 \leq x \leq Uy$ and add Fy
Activate a constraint	indicator, or $a^\top x \leq b + M(1 - y)$
AND / binary product	upper bounds plus $z \geq \sum x_i - (n - 1)$
OR	lower bounds plus $z \leq \sum x_i$
Choose one alternative	$\sum_i y_i = 1$
Binary $\times$ continuous	four bounded-product inequalities
Maximum in minimization	epigraph inequalities $z \geq x_i$
General disjunction	branch binaries plus tight deactivation bounds

The final check is always the same: fix the binary variables to every possible value and verify the resulting feasible set.